

Decorator

miércoles, 6 de mayo de 2026 19:44

Patrón estructural Agrega responsabilidades dinámicamente a un objeto sin modificar su clase y en forma transparente

¿Cuándo Usar?

Si queremos agregar comportamiento adicional a objetos de una clase
Ese comportamiento se puede agregar o quitar dinámicamente en tiempo de ejecución
Usualmente podemos usar herencia pero esta es estática no flexible para este tipo



Ventajas	Desventajas
-Permite mayor flexibilidad que la herencia -Permite agregar funcionalidad incremental	-Mayor cantidad de objetos -> depuración más compleja

Estructura

Misma interface entre componente y decorador

<Component> : interfaz o clase abstracta contiene operaciones

<Concrete Component>(hereda component): provee la funcionalidad básica

<Decorator>(hereda component): clase abstracta(si es la única no), sirve de envoltorio delega las operaciones al objeto referenciado y permite añadir comportamiento

<Concrete Decorator..>(hereda decorator) responsabilidad específica al component que envuelve (efecto,borde,ectc)

Diferencias

Decorator Estructura con solo un siguiente Proposito agregar funcionalidad dinamica	Composite Mantiene una estructura tipo arbol Proposito es componer objetos y tratarlos de manera uniforme
Los dos permiten que un objeto cambie su funcionalidad dinámicamente	Decorator Cambia el algoritmo por fuera del objeto (cambia la piel)
	Strategy Cambia el algoritmo por dentro del objeto (cambia el interior)

